

Module GPS u-blox NEO-6M

Fonctionnement, diagnostic et correction

Projet **Drone à voile**

LilyGO T-Beam V1.1 — ESP32 — Antenne Taoglas ADFGP.25A

Auteur Facundo Arito

Date 20 mai 2026

Fichier source `main/src/drivers/GpsUart.cpp`

0 Contents

1	Vue d'ensemble du système GPS	3
1.1	Rôle du GPS dans le drone	3
1.2	Composants matériels	3
1.3	Flux de données	3
2	Le protocole UBX	3
2.1	Deux langages sur un même fil	3
2.2	Structure d'une trame UBX	4
2.3	Calcul du checksum de Fletcher	4
3	Pourquoi le GPS ne fonctionnait pas	4
3.1	Symptôme observé	4
3.2	Cause racine : la mémoire flash interne du NEO-6M	5
3.3	Cause aggravante : antenne active sans alimentation DC	5
4	Les deux commandes UBX de correction	5
4.1	Commande 1 — CFG-CFG : réinitialisation usine	6
4.1.1	Objectif	6
4.1.2	Structure de la trame	6
4.1.3	Décomposition du payload (13 octets, little-endian)	6
4.1.4	Code source et explication octet par octet	7
4.2	Commande 2 — CFG-ANT : activation de l'antenne active	8
4.2.1	Objectif	8
4.2.2	Structure de la trame	8
4.2.3	Décomposition du payload	8
4.2.4	Code source	9
5	Explication ligne par ligne de GpsUart::begin()	9
5.1	Bloc 1 — Initialisation du parser NMEA personnalisé	10
5.2	Bloc 2 — Ouverture du port série	11
5.3	Bloc 3 — Attente de démarrage du GPS	11
5.4	Bloc 4 — Envoi de CFG-CFG	11
5.5	Bloc 5 — Attente du redémarrage	12
5.6	Bloc 6 — Vidage du buffer	12
5.7	Bloc 7 — Envoi de CFG-ANT	12
6	Fonctionnement de GpsUart::update()	12
7	Guide de diagnostic	14

8 Chronologie de l'initialisation	15
Conclusion	15

1 Vue d'ensemble du système GPS

1.1 Rôle du GPS dans le drone

Le module GPS fournit au drone sa position géographique (latitude, longitude), sa vitesse sur l'eau et son cap. Ces données sont essentielles pour le pilotage automatique et pour le journal de navigation.

1.2 Composants matériels

Matériel impliqué	
Composant	Rôle
LilyGO T-Beam V1.1	Carte principale — intègre un ESP32, un module u-blox NEO-6M et un connecteur u.FL pour antenne externe
u-blox NEO-6M	Récepteur GPS. Communique avec l'ESP32 via UART (<code>Serial1</code>) et accepte des commandes de configuration binaires (protocole UBX)
Taoglas ADFGP.25A	Antenne active externe (patch céramique + LNA intégré). Le LNA nécessite une tension DC transmise sur le câble coaxial
Câble u.FL vers SMA	Connecte le NEO-6M à l'antenne externe

1.3 Flux de données



Le NEO-6M reçoit les signaux radio des satellites, calcule la position et émet des **phrases NMEA** sur son UART. L'ESP32 lit ce flux via `Serial1` et utilise la bibliothèque `TinyGPSPlus` pour en extraire les données utiles.

2 Le protocole UBX

2.1 Deux langages sur un même fil

Le NEO-6M supporte deux protocoles sur son UART :

- **NMEA** — phrases textuelles (ex. `$GPGGA,...`) lues pour extraire la position. C'est le flux normal en fonctionnement.
- **UBX** — protocole binaire propriétaire u-blox, utilisé pour *configurer* le module (activer des fonctions, modifier des paramètres, sauvegarder en mémoire).

Le NEO-6M distingue une trame UBX d'une phrase NMEA grâce aux deux premiers octets : `0xB5 62` pour UBX, le caractère `$` pour NMEA.

2.2 Structure d'une trame UBX

Sync 1	Sync 2	Classe	ID	Longueur (LE)	Payload
0xB5	0x62	1 octet	1 octet	2 octets	N octets

Chaque trame se termine par un **checksum de Fletcher** sur 2 octets (`CK_A`, `CK_B`), calculé sur tous les octets depuis la classe jusqu'à la fin du payload (les deux octets de synchronisation sont exclus).

2.3 Calcul du checksum de Fletcher

```

1 uint8_t CK_A = 0, CK_B = 0;
2 // Parcourir : classe + ID + longueur (2 octets) + payload
3 for (int i = 0; i < N; i++) {
4     CK_A += data[i];
5     CK_B += CK_A;
6 }
7 // CK_A et CK_B sont appended a la fin de la trame

```

Ce checksum garantit l'intégrité de la trame : si un bit est corrompu, le NEO-6M ignore la commande.

3 Pourquoi le GPS ne fonctionnait pas

3.1 Symptôme observé

Le compteur `chars` de `TinyGPSPlus` restait **figé à 10** malgré le GPS alimenté et l'antenne connectée.

Diagnostic : que signifie chars = 10 ?

chars comptabilise **chaque octet reçu** sur **Serial1**. Une trame UBX ACK-ACK (réponse d’acquiescement du NEO-6M) fait exactement **10 octets** :

B5 62 05 01 02 00 06 09 17 6B

B5 62 Sync UBX

05 01 Classe ACK, ID ACK

02 00 Longueur payload = 2

06 09 Écho de la commande acquittée (classe CFG, ID CFG)

17 6B Checksum

Conclusion : le GPS était **bien alimenté et fonctionnel**, il répondait aux commandes UBX, mais n’émettait **aucune phrase NMEA**.

3.2 Cause racine : la mémoire flash interne du NEO-6M

Le NEO-6M possède une **mémoire flash interne** qui persiste la configuration entre les mises sous tension. Une session de configuration antérieure (via le logiciel u-center de u-blox) avait sauvegardé une configuration qui **désactivait toutes les phrases NMEA**.

Cause identifiée

La configuration persistante en flash désactivait la sortie NMEA sur le port UART du NEO-6M. Le module fonctionnait normalement mais n’émettait que des réponses UBX, invisibles pour TinyGPSPlus.

3.3 Cause aggravante : antenne active sans alimentation DC

L’antenne Taoglas ADFGP.25A est une antenne **active** : elle contient un amplificateur LNA qui nécessite une tension DC transmise sur le câble coaxial. Par défaut après une réinitialisation ROM, le superviseur d’antenne du NEO-6M est **désactivé** :

- Aucune tension DC sur le connecteur u.FL $\Rightarrow$ LNA hors tension.
- Le commutateur RF du T-Beam pointe vers le patch céramique interne au lieu de l’antenne externe.
- Résultat : zéro signal GPS même avec l’antenne externe branchée.

4 Les deux commandes UBX de correction

4.1 Commande 1 — CFG-CFG : réinitialisation usine

4.1.1 Objectif

Effacer la configuration sauvegardée en flash et recharger les paramètres d'usine depuis la ROM interne. Après cette commande :

- Le UART1 du NEO-6M repasse à 9 600 bauds.
- Toutes les phrases NMEA standard sont réactivées.
- Le superviseur d'antenne est désactivé (état ROM par défaut).

4.1.2 Structure de la trame

Sync	Classe	ID	Longueur	Payload	CRC
0xB5 62	0x06	0x09	0x0D 00	13 octets	0x71 FE

Classe 0x06 = CFG (configuration), ID 0x09 = sous-commande de gestion de la configuration persistante.

4.1.3 Décomposition du payload (13 octets, little-endian)

Champ (32 bits LE)	Valeur hex	Signification
clearMask	0x0000001F	Efface 5 sections de configuration
saveMask	0x00000000	Ne sauvegarde rien
loadMask	0x0000001F	Recharge depuis la ROM
deviceMask	0x17 (8 bits)	Cible : BBR + Flash + EEPROM

Pourquoi clearMask

Ce champ est un masque de bits : chaque bit correspond à une section de la configuration persistante.

Bit	Valeur hex	Section effacée
0	0x01	ioPort — configuration des ports (UART, SPI, USB)
1	0x02	msgConf — activation des phrases NMEA/UBX par port
2	0x04	infMsg — messages d'information
3	0x08	navConf — paramètres de navigation (mode, SBAS)
4	0x10	rxmConf — mode récepteur
0x01 0x02 0x04 0x08 0x10 = 0x1F		

La section msgConf (bit 1) est la plus critique : c'est elle qui désactivait les phrases NMEA dans notre cas.

4.1.4 Code source et explication octet par octet

```

1 static const uint8_t kCfgCfgReset[] = {
2     0xB5, 0x62,                // Sync UBX -- debut obligato
   ire de toute trame
3     0x06, 0x09,                // Classe CFG (0x06), ID CFG
   (0x09)
4     0x0D, 0x00,                // Longueur payload : 13 octe
   ts (little-endian)
5     0x1F, 0x00, 0x00, 0x00, // clearMask : efface 5 sectio
   ns de config
6     0x00, 0x00, 0x00, 0x00, // saveMask : ne sauvegarde r
   ien
7     0x1F, 0x00, 0x00, 0x00, // loadMask : recharge depuis
   la ROM
8     0x17,                      // deviceMask : BBR(1)+Flash
   (2)+EEPROM(4)+SPI(16)
9     0x71, 0xFE                // Checksum Fletcher (CK_A=0
   x71, CK_B=0xFE)
10 };

```


Pourquoi `static const` ?

`static` dans une fonction signifie que le tableau est alloué **une seule fois en mémoire flash** (section `.rodata`), et non sur la pile à chaque appel. Sur un microcontrôleur avec peu de RAM comme l'ESP32, c'est une bonne pratique pour les constantes volumineuses.

4.2 Commande 2 — CFG-ANT : activation de l'antenne active

4.2.1 Objectif

Activer le **superviseur d'antenne** du NEO-6M, qui accomplit trois actions :

1. Fournit une tension DC sur le câble coaxial pour alimenter le LNA de l'antenne Taoglas.
2. Commande le commutateur RF du T-Beam (via la broche `ANT_FLAG` du NEO-6M) pour sélectionner l'antenne externe u.FL.
3. Active la détection de court-circuit pour protéger le module.

4.2.2 Structure de la trame

Sync	Classe	ID	Longueur	Payload	CRC
0xB5 62	0x06	0x13	0x04 00	4 octets	0x47 57

Classe 0x06 = CFG, ID 0x13 = ANT (configuration de l'antenne).

4.2.3 Décomposition du payload

Champ (16 bits)	Valeur	Signification
flags	0x001B	Voir tableau ci-dessous
pins	0x000F	Mapping broches — valeur par défaut NEO-6M

Décomposition de flags = 0x001B

Bit	Nom	Fonction
0	svcs	Alimentation DC + commutateur RF vers antenne externe
1	scd	Détection de court-circuit
3	pdwn0nSCD	Coupe l'alimentation si court-circuit détecté
4	recovery	Rétablissement automatique après court-circuit
0x01 0x02 0x08 0x10 = 0x1B		

4.2.4 Code source

```

1 static const uint8_t kCfgAnt[] = {
2     0xB5, 0x62,    // Sync UBX
3     0x06, 0x13,    // Classe CFG (0x06), ID ANT (0x13)
4     0x04, 0x00,    // Longueur payload : 4 octets
5     0x1B, 0x00,    // flags (LE) : svcs + scd + pdwn0nSCD +
                      recovery
6     0x0F, 0x00,    // pins (LE) : mapping broches par defa
                      ut NEO-6M
7     0x47, 0x57     // Checksum Fletcher
8 };

```

5 Explication ligne par ligne de GpsUart::begin()

Voici le code complet de la fonction, suivi d'une explication détaillée de chaque bloc.

```

1 void GpsUart::begin() {
2     gsvTotal_.begin(gps_, "GPGSV", 3);
3     Serial1.begin(BoardConfig::GPS_BAUD_RATE, SERIAL_8N1,
4                   BoardConfig::GPS_RX_PIN, BoardConfig::
5 GPS_TX_PIN);
6     DBG("GPS", "Serial1 started %u baud RX=GPIO%d TX=GPIO%
7 d",
8         (unsigned)BoardConfig::GPS_BAUD_RATE,
9         BoardConfig::GPS_RX_PIN, BoardConfig::GPS_TX_PIN);

```

```

9      delay(500);    // attendre fin du boot interne du NEO-6
M
10
11      static const uint8_t kCfgCfgReset[] = {
12          0xB5, 0x62, 0x06, 0x09, 0x0D, 0x00,
13          0x1F, 0x00, 0x00, 0x00,
14          0x00, 0x00, 0x00, 0x00,
15          0x1F, 0x00, 0x00, 0x00,
16          0x17,
17          0x71, 0xFE
18      };
19      Serial1.write(kCfgCfgReset, sizeof(kCfgCfgReset));
20      DBG("GPS", "CFG-CFG sent -- factory reset to ROM defaults");
21      delay(1500);    // attendre redemarrage apres reinitiali
sation
22
23      while (Serial1.available()) Serial1.read();    // vider
le buffer
24
25      static const uint8_t kCfgAnt[] = {
26          0xB5, 0x62, 0x06, 0x13, 0x04, 0x00,
27          0x1B, 0x00,
28          0x0F, 0x00,
29          0x47, 0x57
30      };
31      Serial1.write(kCfgAnt, sizeof(kCfgAnt));
32      delay(100);
33      DBG("GPS", "CFG-ANT sent -- active antenna supervisor
+ bias enabled");
34  }

```

5.1 Bloc 1 — Initialisation du parser NMEA personnalisé

```

1  gsvTotal_.begin(gps_, "GPGSV", 3);

```

TinyGPSPlus ne décode pas toutes les phrases NMEA automatiquement. La phrase GPGSV (satellites en vue) n'est pas traitée par défaut : il faut déclarer explicitement quel champ extraire.

`gsvTotal_` est un objet `TinyGPSCustom`. Cette ligne lui dit : dans les phrases `GPGSV`, extraire le **champ numéro 3** (le nombre total de satellites en vue). Ce champ devient lisible via `gsvTotal_.value()`.

5.2 Bloc 2 — Ouverture du port série

```
1 Serial1.begin(BoardConfig::GPS_BAUD_RATE, SERIAL_8N1,
2               BoardConfig::GPS_RX_PIN, BoardConfig::
   GPS_TX_PIN);
```

Configure le port UART1 de l'ESP32 pour communiquer avec le NEO-6M :

- `GPS_BAUD_RATE` = 9 600 bauds — vitesse par défaut ROM du NEO-6M.
- `SERIAL_8N1` — 8 bits de données, pas de parité, 1 bit de stop.
- Les broches GPIO sont définies dans `BoardConfig.h` et correspondent au câblage du T-Beam.

5.3 Bloc 3 — Attente de démarrage du GPS

```
1 delay(500);
```

Le NEO-6M a besoin d'environ 100 à 400 ms pour terminer sa séquence de mise sous tension. Sans ce délai, les commandes UBX envoyées immédiatement pourraient être ignorées. On attend 500 ms pour une marge confortable.

5.4 Bloc 4 — Envoi de CFG-CFG

```
1 Serial1.write(kCfgCfgReset, sizeof(kCfgCfgReset));
```

Envoie les 21 octets de la trame CFG-CFG sur le UART. `sizeof()` est évalué par le compilateur : sa valeur est exacte et ne peut pas être oubliée ou erronée.

Le NEO-6M reçoit la trame, vérifie le checksum, puis :

1. Efface les sections spécifiées en flash et BBR.
2. Recharge ces sections depuis sa ROM (paramètres d'usine).
3. Envoie une trame ACK-ACK de confirmation.

4. Redémarre son moteur de navigation interne.

5.5 Bloc 5 — Attente du redémarrage

```
1 delay(1500);
```

La réinitialisation ROM prend entre 500 ms et 1 200 ms. On attend 1 500 ms pour s'assurer que le module est de nouveau prêt avant d'envoyer CFG-ANT.

5.6 Bloc 6 — Vidage du buffer

```
1 while (Serial1.available()) Serial1.read();
```

Pendant le délai de 1 500 ms, le GPS a pu envoyer des données sur le UART (ACK-ACK, premières phrases NMEA de démarrage). Ces données s'accumulent dans le buffer de l'ESP32. On les lit et on les jette pour repartir sur une base propre avant CFG-ANT.

5.7 Bloc 7 — Envoi de CFG-ANT

```
1 Serial1.write(kCfgAnt, sizeof(kCfgAnt));  
2 delay(100);
```

Envoie les 12 octets de la trame CFG-ANT. Le NEO-6M l'applique immédiatement :

1. Alimentation DC activée sur u.FL $\Rightarrow$ LNA Taoglas alimenté.
2. Commutateur RF du T-Beam basculé vers l'antenne externe.
3. Détection de court-circuit activée.

Le `delay(100)` laisse le temps au module d'appliquer la configuration et de renvoyer l'ACK avant la fin de la fonction.

6 Fonctionnement de `GpsUart::update()`

La fonction `update()` est appelée à chaque cycle de la boucle principale (environ 50 fois par seconde). Elle lit les octets disponibles sur le UART et les passe au parser `TinyGPSPlus`.

```

1 void GpsUart::update() {
2     while (Serial1.available()) {
3         char c = Serial1.read();
4         gps_.encode(c);           // (1) passe l'octet
        au parser
5
6         if (c == '\n') {         // (2) fin de phrase
            NMEA
7             lineBuf_[lineLen_] = '\0';
8             memcpy(completedLine_, lineBuf_, lineLen_ + 1)
9         ;
10            lineLen_ = 0;
11        } else if (c != '\r' && lineLen_ < sizeof(lineBuf_
12        ) - 1) {
13            lineBuf_[lineLen_++] = c; // (3) accumule les c
14            aracteres
15        }
16    }
17
18    const bool nowValid = gps_.location.isValid(); // (4)
        fix GPS actif ?
19
20    if (nowValid && !prevValid_) { DBG("GPS", "FIX ACQUIRE
21    D ..."); }
22    if (!nowValid && prevValid_) { DBG("GPS", "FIX LOST");
23    }
24    prevValid_ = nowValid;
25
26    pos_.valid      = nowValid;           // (5) mise a jour
        structure publique
27    pos_.lat        = nowValid ? gps_.location.lat() :
28    0.0;
29    pos_.lon        = nowValid ? gps_.location.lng() :
30    0.0;
31    pos_.speedKmph   = gps_.speed.isValid()
32                      ? (float)gps_.speed.kmph() : 0.0f
33    ;
34    pos_.courseDeg   = gps_.course.isValid()
35                      ? (float)gps_.course.deg() : 0.0f
36    ;
37 }

```

```

28     pos_.satellites = gps_.satellites.isValid()
29                     ? (uint8_t)gps_.satellites.value()
30     : 0;
31     pos_.hdop       = gps_.hdop.isValid()
32                     ? (float)gps_.hdop.hdop()      : 99.9
33     f;
34     pos_.ageMs      = gps_.location.isValid()
35                     ? gps_.location.age() : 0xFFFFFFFF
36     UL;
37 }

```

1. **gps_.encode(c)** — Le cœur du parser. TinyGPSPlus reçoit un octet à la fois et reconstruit les phrases NMEA en interne. Quand une phrase complète et valide est reçue, il met à jour ses champs internes (`location`, `speed`, `satellites`, etc.).
2. **Capture de la phrase complète** — Quand le caractère `\n` (fin de ligne NMEA) est reçu, on copie le buffer dans `completedLine_` pour permettre l’affichage de la dernière phrase NMEA lors du diagnostic.
3. **Accumulation caractère par caractère** — Les caractères sont accumulés dans `lineBuf_`. Le `\r` (retour chariot) est ignoré : le protocole NMEA utilise `CR+LF` comme fin de ligne, mais seul `LF` délimite la phrase.
4. **gps_.location.isValid()** — Retourne `true` uniquement si une phrase NMEA valide avec fix actif a été reçue récemment. Un fix froid (premier démarrage) prend typiquement 30 secondes à 2 minutes à l’extérieur avec une bonne vue du ciel.
5. **Mise à jour de pos_** — La structure publique est rafraîchie à chaque appel. Un champ invalide retourne une valeur de sécurité : `0.0` pour les coordonnées, `99.9` pour HDOP (valeur maximale = pas de fix), `0xFFFFFFFF` pour l’âge (signifiant donnée absente).

7 Guide de diagnostic

Symptôme	Cause probable	Action
chars figé à 0	UART mal câblé, broches TX/RX échangées, ou GPS non alimenté	Vérifier câblage et alimentation

Symptôme	Cause probable	Action
<code>chars</code> figé à 10	Config flash corrompue — NMEA désactivé	Déjà corrigé dans <code>begin()</code> par CFG-CFG
<code>chars</code> monte, <code>visible</code> = 0	Obstruction physique (toit, bâtiment)	Aller à l'extérieur, ciel dégagé
<code>chars</code> monte, <code>visible</code> > 0, pas de fix	Temps d'acquisition froid normal	Attendre 1 à 3 min à l'extérieur
<code>chars</code> monte, <code>visible</code> = 0 même dehors	LNA non alimenté ou mauvais connecteur	Vérifier CFG-ANT, câble u.FL et sertissage
Fix acquis puis perdu régulièrement	HDOP élevé, obstruction partielle	Dégager la vue du ciel (mât, pont)

8 Chronologie de l'initialisation

Temps (ms)	Événement
0	<code>Serial1.begin()</code> — UART ESP32 ouvert à 9 600 bauds
0 – 500	Délai de démarrage — le NEO-6M termine son boot interne
500	Envoi de CFG-CFG — demande de réinitialisation ROM
≈ 501	Le NEO-6M répond ACK-ACK (10 octets)
501 – 2000	Délai de redémarrage — rechargement de la config ROM
2000	Vidage du buffer UART
2000	Envoi de CFG-ANT — activation du superviseur d'antenne
≈ 2100	Fin de <code>begin()</code> — le GPS est prêt à émettre du NMEA
2100+	<code>update()</code> appelé en boucle — décodage des phrases NMEA
≈ 60 000 – 180 000	Premier fix GPS (typique en extérieur, ciel dégagé)

8 Conclusion

Le dysfonctionnement du GPS était dû à la persistance d'une configuration corrompue dans la mémoire flash interne du NEO-6M, qui avait désactivé toutes les sorties NMEA. La correction consiste à envoyer systématiquement, à chaque démarrage, une commande

de réinitialisation usine (**CFG-CFG**) suivie de l'activation du superviseur d'antenne active (**CFG-ANT**).

Ces deux commandes sont intégrées dans `GpsUart::begin()`, ce qui garantit un comportement correct indépendamment de toute session de configuration antérieure effectuée avec u-center ou tout autre outil.

État actuel du système

- ✓ Phrases NMEA réactivées systématiquement à chaque boot.
- ✓ Antenne active Taoglas ADFGP.25A alimentée correctement.
- ✓ Commutateur RF T-Beam basculé vers l'antenne externe.
- ✓ Fix GPS confirmé lors des tests en extérieur.